

Labor: Java Grundlagen

Java-Programmierung · 3 Stunden · Einzelarbeit + Partnerarbeit

💡 *Ihr habt C gelernt: Structs, Pointer, Funktionen, MQTT. Java ist keine neue Welt — es ist dieselbe Welt mit anderen Werkzeugen. Heute baut ihr das MQTT-Schema, das ihr selbst entworfen habt, als Java-Klasse nach. Was dabei anders ist — und warum das besser ist — seht ihr selbst.*

Ablauf heute:

Teil 1: Umgebung + Ausgabe (40 min)
Teil 2: Klassen und Konstruktoren (25 min)
Teil 3: Spezifikation + Implementierung (60 min)
Teil 4: Reflexion (20 min)

Ziel am Ende:

- ✓ Java-Umgebung läuft in Codespaces
- ✓ Eigene Klasse mit Konstruktor + Validierung
- ✓ Spec vor Code — auch in Java
- ✓ Testvertrag mit println-Beweis

TEIL 1 · Umgebung + Ausgabe (40 min)

💡 *Kein Framework, kein Build-Tool, keine IDE-Magie. Erst wenn ihr seht, was `javac` und `java` tun — Schritt für Schritt — macht das Drumherum später Sinn.*

1.1 Codespaces starten

Öffnet das Kurs-Repository in GitHub Codespaces. Sobald die Umgebung läuft, macht folgendes:

```
# Terminal in Codespaces:
java --version           # sollte Java 25 zeigen
mkdir session01 && cd session01
touch Hallo.java         # leere Datei anlegen
javac Hallo.java         # kompilieren — noch leer, kein Fehler
java Hallo               # ausführen — noch nichts passiert
```

Was habt ihr beobachtet? Was hat javac produziert?

⚠ java Hallo — nicht java Hallo.java. Das ist ein häufiger Fehler: java führt kompilierten Bytecode aus, keine Quelldatei. Die Dateiergung .class wird weggelassen.

1.2 System.out.println — der Feedback-Loop

💡 *Ohne Ausgabe ist euer Programm eine Blackbox. Bevor ihr irgendeine Klasse schreibt, braucht ihr dieses Werkzeug — genau wie printf in C.*

Schreibt folgendes in Hallo.java — zuerst auf Papier, dann tippt ihr es:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hallo, Java!");  
        int zahl = 42;  
        System.out.println("Die Zahl ist: " + zahl);  
        double temp = 19.3;  
        System.out.println("Temperatur: " + temp);  
    }  
}
```

Bevor ihr kompiliert: Was werdet ihr auf der Konsole sehen? Schreibt die erwartete Ausgabe auf:

Jetzt kompilieren und ausführen. Hat die Ausgabe gestimmt?

Übung: System.err.println

Fügt eine weitere Zeile ein:

```
System.err.println("Das ist eine Fehlermeldung");
```

Was ist der Unterschied zwischen System.out und System.err? Wann würdet ihr welchen nutzen?

1.3 Typen: C und Java im Vergleich

💡 *Ihr kennt diese Typen aus C. In Java heißen sie fast gleich — aber ein Unterschied ist wichtig, und den sollt ihr jetzt herausfinden.*

Füllt die Tabelle aus — aus dem Gedächtnis, kein Nachschlagen:

C-Typ	Java-Typ	Unterschied / Anmerkung
int		
float		
double		
char		
— (kein Äquivalent)		
char * / Array		

Welcher Typ in Java hat kein direktes Äquivalent in C — und warum ist das ein Vorteil?

TEIL 2 · Klassen und Konstruktoren (25 min)

💡 *Ihr habt im MQTT-Labor ein Struct entworfen. Heute baut ihr es als Java-Klasse nach — zuerst ohne Verhalten, dann mit. Der Unterschied ist der Punkt.*

2.1 Vom C-Struct zur Java-Klasse

Schreibt euer SensorReading-Struct aus dem MQTT-Labor auf Papier — aus dem Gedächtnis. Alle Felder, die ihr damals definiert habt:

Jetzt dasselbe als Java-Klasse — nur Felder, noch kein Konstruktor, noch keine Methoden. Neue Datei: SensorReading.java

```
public class SensorReading {  
    // Felder hier – was braucht eine Sensornachricht?  
    public int seq;  
    public String stationId;  
    public String timestamp;  
    public double temperatureC;  
    public double humidityPct;  
}
```

Was kann diese Klasse, was euer C-Struct nicht konnte? (Ehrliche Antwort: noch nichts. Aber was könnte sie können?)

--

2.2 Konstruktor: Initialisierung mit Garantien

Problem: Wie stellt ihr sicher, dass temperatureC nie unter -273.15 liegt? In C war das eine externe Prüfung — der Aufrufer musste daran denken. In Java kann die Klasse das selbst erzwingen.

💡 Ein Konstruktor hat keinen Rückgabewert — nicht mal void. Er heißt genauso wie die Klasse. Er wird aufgerufen wenn ein Objekt mit new erzeugt wird.

Erweiterung: fügt einen Konstruktor in SensorReading.java ein:

```
public SensorReading(int seq, String stationId, double temperatureC, double humidityPct) {  
    this.seq = seq;  
    this.stationId = stationId;  
    this.humidityPct = humidityPct;  
    // Validierung:  
    if (temperatureC < -273.15) {
```

```
        System.err.println("Ungültige Temperatur: " + temperatureC);
        this.temperatureC = -273.15; // Fallback
    } else {
        this.temperatureC = temperatureC;
    }
}
```

Was macht `this.temperatureC = temperatureC` hier? Warum braucht ihr `this`?

Testet den Konstruktor in einer Main-Klasse — zuerst auf Papier, dann tippt ihr. Was erwartet ihr auf der Konsole?

```
public class Main {
    public static void main(String[] args) {
        SensorReading r1 = new SensorReading(1, "S1", 19.3, 64.2);
        System.out.println("Temperatur: " + r1.temperatureC);

        SensorReading r2 = new SensorReading(2, "S1", -999.0, 64.2);
        System.out.println("Temperatur: " + r2.temperatureC); // Was steht
hier?
    }
}
```

Erwartete Ausgabe (vor dem Ausführen):

Tatsächliche Ausgabe:

TEIL 3 · Spezifikation + Implementierung (60 min)

 Jetzt arbeitet ihr in Zweiergruppen. Kein Code in Phase 3.1 — erst die Spec, dann der Konstruktor, dann der Beweis.

3.1 Spezifikation: Vor dem ersten Zeichen Code

Schätzt zuerst — bevor ihr lest, was zu tun ist:

Aufgabe	Schätzung (min)	Tatsächlich (min)	KI genutzt?
Spec schreiben (3.1)			<i>ja / nein</i>
Konstruktor implementieren (3.2)			<i>ja / nein</i>
Testvertrag ausführen (3.3)			<i>ja / nein</i>
Methode print() hinzufügen			<i>ja / nein</i>
KI-Aufgabe + Vergleich (3.4)			<i>ja / nein</i>

Jetzt definiert ihr den Vertrag für euren Konstruktor. Kein Code — nur Prosa und Tabellen.

SensorReading(int seq, String stationId, double temperatureC, double humidityPct)	
Aufgabe (ein Satz)	
Randfall: temperatureC < -273.15	
Randfall: humidityPct < 0 oder > 100	
Randfall: stationId == null	
Was darf der Aufrufer annehmen?	
Was darf der Aufrufer nicht annehmen?	

3.2 Implementierung: Konstruktor nach Spec

Erweitert euren Konstruktor so, dass er alle drei Randfälle aus der Spec behandelt. Haltet euch an eure Spec — nicht an das was bequem zu implementieren ist.

 Wenn ihr merkt, dass eure Spec eine Lücke hat: aktualisiert zuerst die Spec, dann den Code. Nicht umgekehrt.

3.3 Testvertrag: Beweis durch Ausgabe

Schreibt Tests in eurer Main-Klasse. Für jeden Testfall: erwartet ihr erst, dann führt ihr aus, dann tragt ihr das Ergebnis ein.

Testfall	Erwartetes Ergebnis	Bestanden?
new SensorReading(1, "S1", 19.3, 64.2) — Normalfall		✓ / ✗
new SensorReading(2, "S1", -999.0, 64.2) — Temperatur unter Minimum		✓ / ✗
new SensorReading(3, "S1", 22.0, 150.0) — Luftfeuchte über 100		✓ / ✗
new SensorReading(4, null, 22.0, 64.0) — Station null		✓ / ✗
System.out.println gibt die Werte korrekt aus		✓ / ✗

3.4 KI-Einsatz: Methode print() mit und ohne Spec

Fügt eine Methode print() hinzu, die alle Felder lesbar ausgibt. Zuerst versucht ihr einen vagen Prompt — dann einen spec-basierten. Vergleicht die Ergebnisse.

✗ Vager Prompt	✓ Spec-basierter Prompt
Schreib eine Java-Methode die ein SensorReading ausgibt.	Schreib eine Java-Methode mit dieser Signatur: public void print() Regeln: - Keine Parameter, kein Rückgabewert - Ausgabe auf System.out (nicht err) - Format: "S1 seq=1 19.3°C 64.2%" - Felder: stationId, seq, temperatureC, humidityPct - Keine println in anderen Methoden aufrufen

Mein Prompt (vor dem Abschicken aufschreiben):

Was hat die KI nicht richtig gemacht — was stimmte nicht mit eurer Spec überein?

TEIL 4 · Reflexion (20 min)

 *Ganze Sätze. Bezieht euch auf eure eigene Arbeit heute — nicht auf das was theoretisch richtig wäre.*

Vergleicht euren Konstruktor mit dem C-Struct aus dem MQTT-Labor. Was kann der Konstruktor, was das Struct nicht konnte — und was kostet euch das?

Wo hat eure Spec eine Lücke gehabt? Wann habt ihr das gemerkt — beim Schreiben des Konstruktors oder erst beim Testen?

Vergleicht die Ausgabe des vagen Prompts mit der Ausgabe des spec-basierten Prompts. Was war konkret anders — und warum?

Zeitkontrolle Tatsächliche Zeiten eintragen

Tragt jetzt die tatsächlichen Zeiten in die Schätztable aus 3.1 ein. Was war die größte Abweichung?

Abschluss-Checkpoint

- ☐ SensorReading.java — kompiliert und ausführbar
- ☐ Konstruktor behandelt alle drei Randfälle aus der Spec
- ☐ Testvertrag-Tabelle ausgefüllt — alle Testfälle ausgeführt
- ☐ Spec-Tabelle vollständig — kein leeres Feld
- ☐ Prompt-Vergleich: beide Prompts dokumentiert, Unterschied beschrieben
- ☐ Zeitschätzung: Ist-Zeiten eingetragen und kommentiert
- ☐ Reflexion: alle drei Fragen in ganzen Sätzen beantwortet
- ☐ Mündlicher Checkpoint: Ihr könnt jede Zeile eures Konstruktors erklären

Ein Programm das kompiliert ist kein Beweis. Ein Testvertrag der bestanden hat, ist einer.